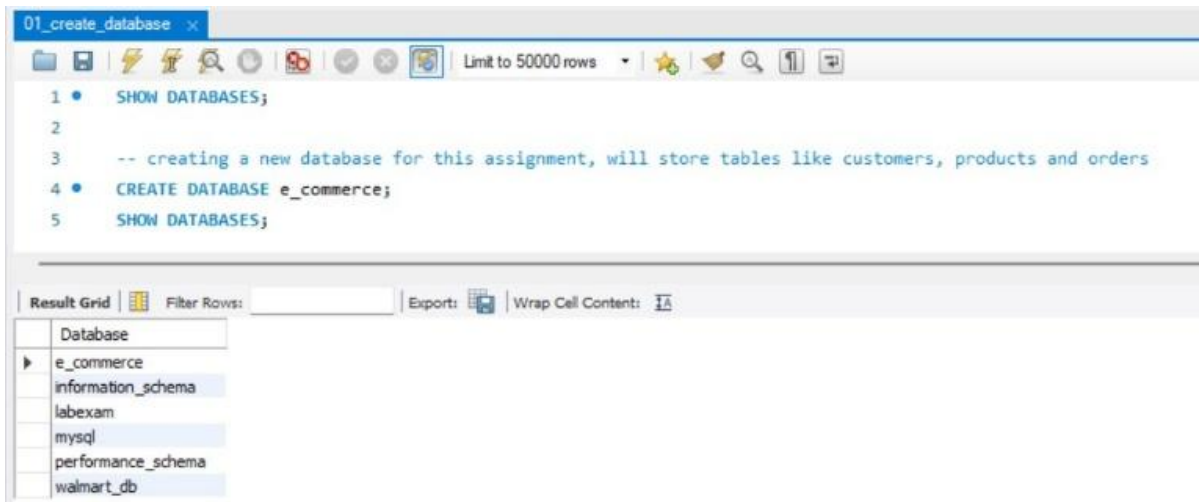


Name : More Shrinivas Balaji

Topic : Basic SQL Assignment

1)Create Database

- Created a database named e_commerce to manage all tables.



The screenshot shows a SQL IDE window titled '01_create_database'. The SQL editor contains the following code:

```
1 • SHOW DATABASES;
2
3 -- creating a new database for this assignment, will store tables like customers, products and orders
4 • CREATE DATABASE e_commerce;
5 SHOW DATABASES;
```

The interface includes a toolbar at the top with icons for file operations and a 'Limit to 50000 rows' dropdown. Below the editor is a 'Result Grid' section with a 'Filter Rows' input and an 'Export' button. The 'Result Grid' displays a list of databases: 'Database', 'e_commerce', 'information_schema', 'labexam', 'mysql', 'performance_schema', and 'walmart_db'. The 'e_commerce' database is selected and expanded.

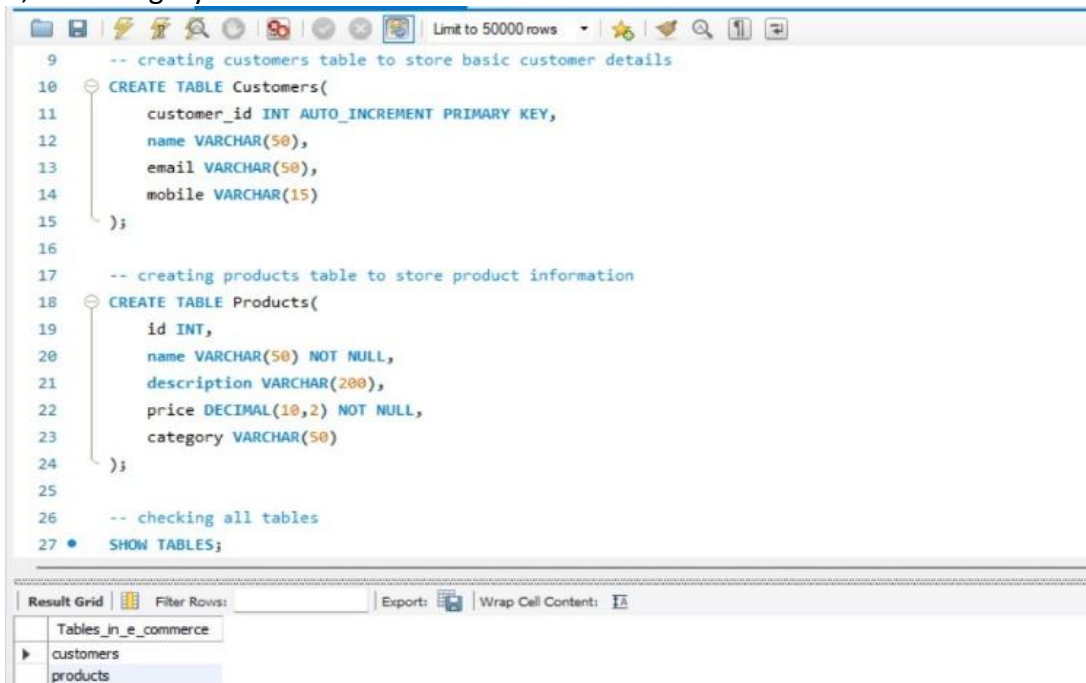
2)Table Creation

- Customers Table**

Created the Customers table to store customer details such as name, email and mobile number.

- Products Table**

Created the Products table to maintain product information including id, name, description, price, and category.



The screenshot shows a SQL IDE window with the following code in the editor:

```
9 -- creating customers table to store basic customer details
10 CREATE TABLE Customers(
11     customer_id INT AUTO_INCREMENT PRIMARY KEY,
12     name VARCHAR(50),
13     email VARCHAR(50),
14     mobile VARCHAR(15)
15 );
16
17 -- creating products table to store product information
18 CREATE TABLE Products(
19     id INT,
20     name VARCHAR(50) NOT NULL,
21     description VARCHAR(200),
22     price DECIMAL(10,2) NOT NULL,
23     category VARCHAR(50)
24 );
25
26 -- checking all tables
27 • SHOW TABLES;
```

The interface includes a toolbar at the top with icons for file operations and a 'Limit to 50000 rows' dropdown. Below the editor is a 'Result Grid' section with a 'Filter Rows' input and an 'Export' button. The 'Result Grid' displays a list of tables in the 'e_commerce' database: 'Tables_in_e_commerce', 'customers', and 'products'. The 'customers' table is selected and expanded.

3)Table Modifications

a) Add not null on name and email in the Customers table

- Applied NOT NULL constraint on name and email to ensure mandatory data entry

The screenshot shows a SQL IDE window titled "03_alter_customers_table* x". The SQL editor contains the following queries:

```
29 -- making name and email not null so empty values are not allowed
30 • ALTER TABLE Customers
31   MODIFY name VARCHAR(50) NOT NULL;
32
33 • ALTER TABLE Customers
34   MODIFY email VARCHAR(50) NOT NULL;
35
36 • DESCRIBE Customers;
```

Below the editor is a "Result Grid" showing the table structure after the modifications:

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	auto_increment
name	varchar(50)	NO		NULL	
email	varchar(50)	NO		NULL	
mobile	varchar(15)	YES		NULL	

b) Add unique key on email in the Customers table

- Added UNIQUE constraint on email to prevent duplicate records.

The screenshot shows the same SQL IDE window with the following queries:

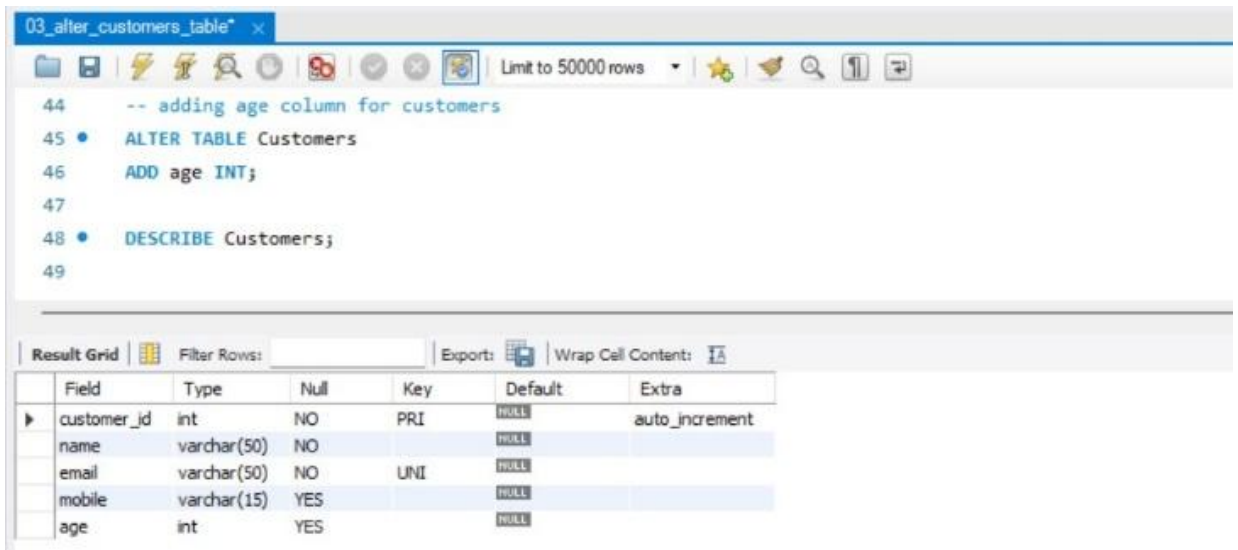
```
37
38 -- adding unique constraint on email to avoid duplicate emails
39 • ALTER TABLE Customers
40   ADD CONSTRAINT unique_email UNIQUE(email);
41
42 • DESCRIBE Customers;
```

The "Result Grid" shows the updated table structure:

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	auto_increment
name	varchar(50)	NO		NULL	
email	varchar(50)	NO	UNI	NULL	
mobile	varchar(15)	YES		NULL	

c) Add column age in the Customers table

- Added a new column 'age'.

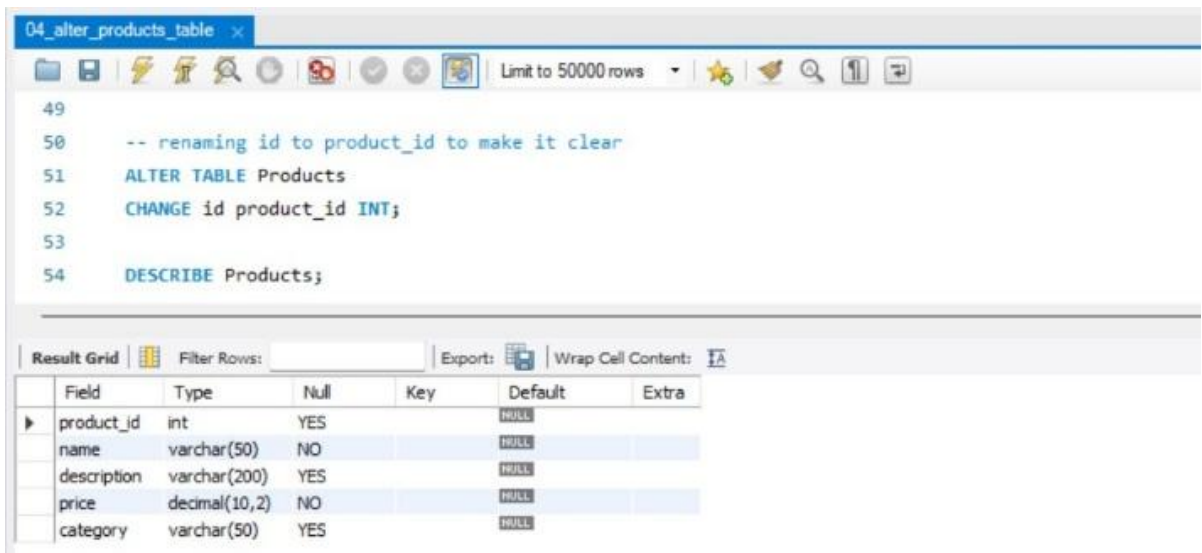


```
03_alter_customers_table* x
-- adding age column for customers
ALTER TABLE Customers
ADD age INT;
DESCRIBE Customers;
```

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	auto_increment
name	varchar(50)	NO		NULL	
email	varchar(50)	NO	UNI	NULL	
mobile	varchar(15)	YES		NULL	
age	int	YES		NULL	

d) Change column name from id to product_id in the Products table;

- Renamed column id to product_id for better readability and consistency.

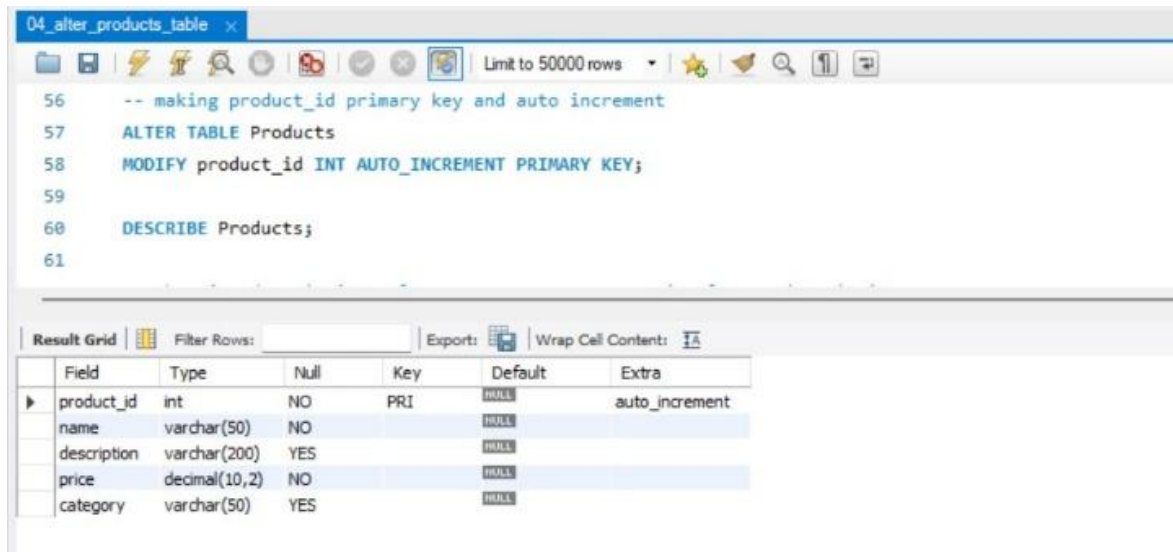


```
04_alter_products_table x
-- renaming id to product_id to make it clear
ALTER TABLE Products
CHANGE id product_id INT;
DESCRIBE Products;
```

Field	Type	Null	Key	Default	Extra
product_id	int	YES		NULL	
name	varchar(50)	NO		NULL	
description	varchar(200)	YES		NULL	
price	decimal(10,2)	NO		NULL	
category	varchar(50)	YES		NULL	

e) Add primary key and auto increment on product_id in the Products table

- Defined product_id as primary key and enabled auto increment.

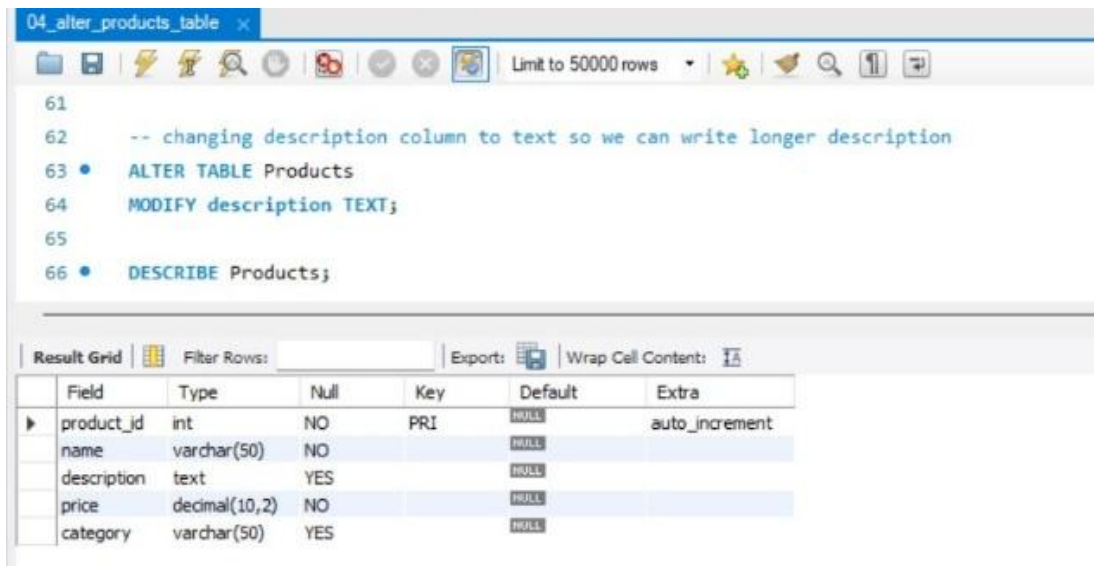


```
56 -- making product_id primary key and auto increment
57 ALTER TABLE Products
58 MODIFY product_id INT AUTO_INCREMENT PRIMARY KEY;
59
60 DESCRIBE Products;
61
```

Field	Type	Null	Key	Default	Extra
product_id	int	NO	PRI	NULL	auto_increment
name	varchar(50)	NO		NULL	
description	varchar(200)	YES		NULL	
price	decimal(10,2)	NO		NULL	
category	varchar(50)	YES		NULL	

f) Change datatype of description from varchar to text in the Products table

- Modified description datatype from VARCHAR to TEXT to support larger content.

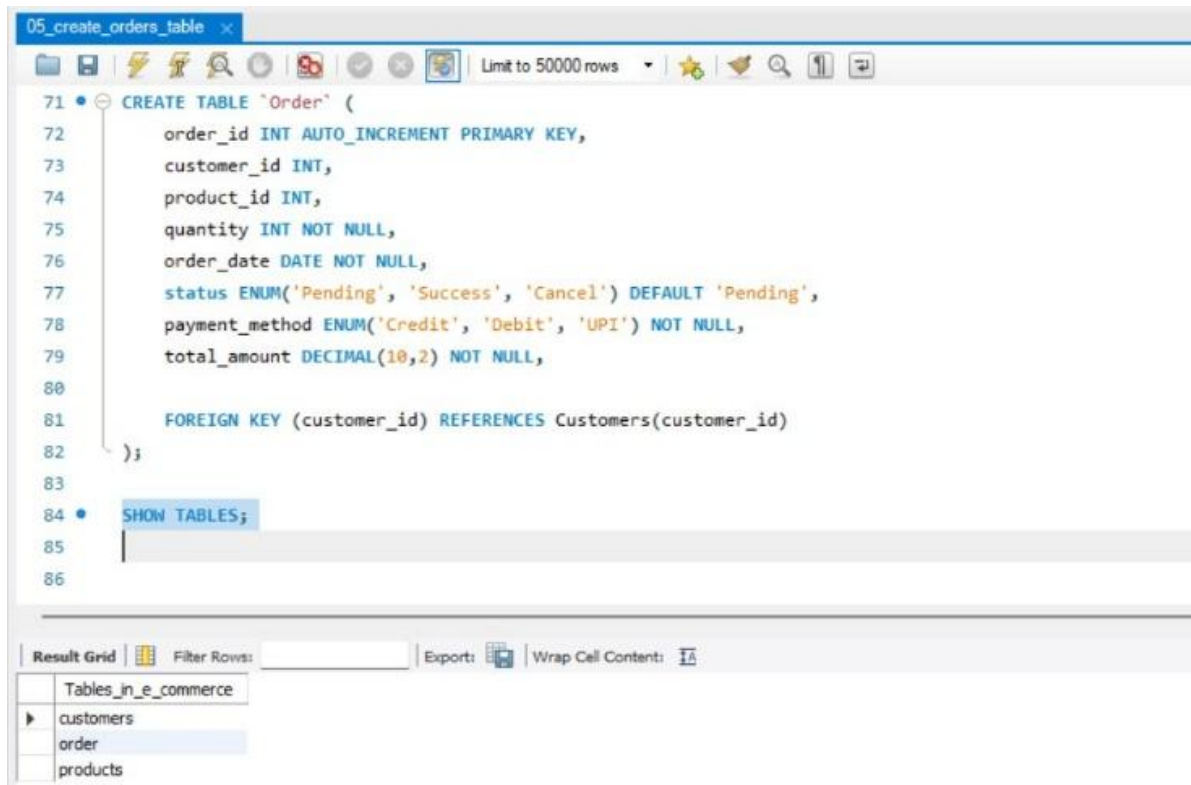


```
61
62 -- changing description column to text so we can write longer description
63 • ALTER TABLE Products
64 MODIFY description TEXT;
65
66 • DESCRIBE Products;
```

Field	Type	Null	Key	Default	Extra
product_id	int	NO	PRI	NULL	auto_increment
name	varchar(50)	NO		NULL	
description	text	YES		NULL	
price	decimal(10,2)	NO		NULL	
category	varchar(50)	YES		NULL	

4)Orders Table Creation

- Created Orders table to store transaction details including id,customer, product, quantity, order date,status,total_amount and payment method.



The screenshot shows a database IDE window titled '05_create_orders_table'. The main editor contains SQL code for creating a table named 'Order'. The code includes fields for order_id (primary key), customer_id, product_id, quantity, order_date, status (with a default of 'Pending'), payment_method, and total_amount. It also includes a foreign key constraint linking customer_id to the Customers table. Below the code editor, the 'Result Grid' tab is active, showing a list of tables in the 'Tables_in_e-commerce' database: customers, order, and products. The 'order' table is currently selected.

```
71 • CREATE TABLE `Order` (  
72     order_id INT AUTO_INCREMENT PRIMARY KEY,  
73     customer_id INT,  
74     product_id INT,  
75     quantity INT NOT NULL,  
76     order_date DATE NOT NULL,  
77     status ENUM('Pending', 'Success', 'Cancel') DEFAULT 'Pending',  
78     payment_method ENUM('Credit', 'Debit', 'UPI') NOT NULL,  
79     total_amount DECIMAL(10,2) NOT NULL,  
80  
81     FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)  
82 );  
83  
84 • SHOW TABLES;  
85  
86
```

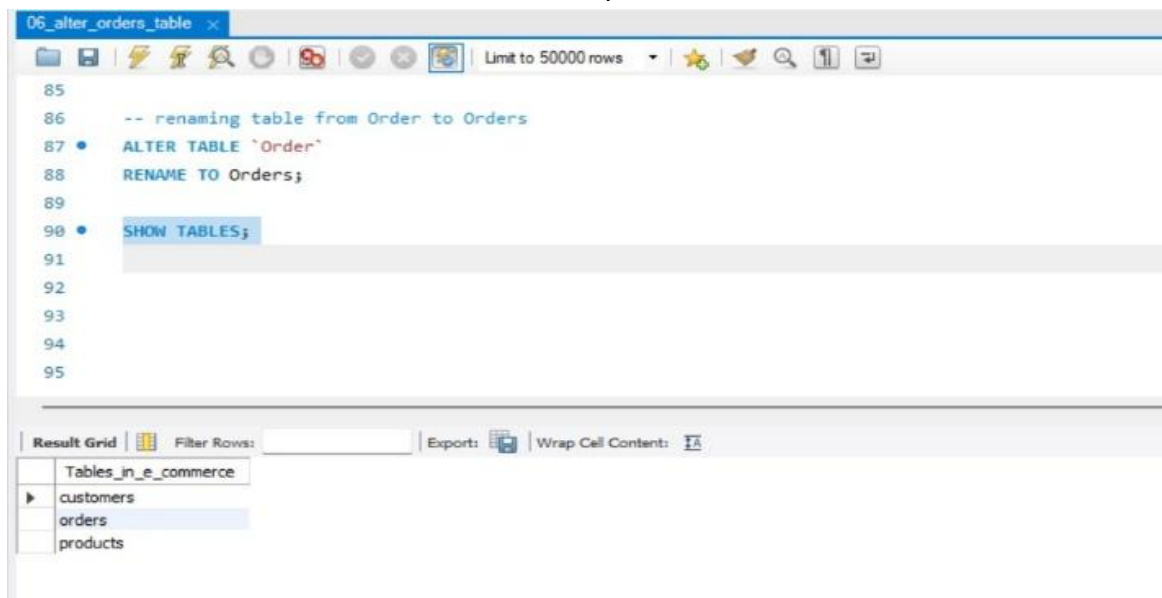
Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

Tables_in_e-commerce
customers
order
products

5)Orders Table Modification

a) Change table name Order -> Orders

- Renamed table from Order to Orders for clarity.



The screenshot shows a database IDE window titled '06_alter_orders_table'. The main editor contains SQL code for renaming the table. The code includes a comment '-- renaming table from Order to Orders', followed by the SQL command 'ALTER TABLE `Order` RENAME TO Orders;'. Below the code editor, the 'Result Grid' tab is active, showing a list of tables in the 'Tables_in_e-commerce' database: customers, orders, and products. The 'orders' table is currently selected.

```
85  
86 -- renaming table from Order to Orders  
87 • ALTER TABLE `Order`  
88     RENAME TO Orders;  
89  
90 • SHOW TABLES;  
91  
92  
93  
94  
95
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: [IA](#)

Tables_in_e-commerce
customers
orders
products

b) Set default value pending in status.

- Set default value of status as 'Pending' to handle new orders.

The screenshot shows a database IDE window titled "06_alter_orders_table". The SQL editor contains the following code:

```
91
92 -- setting default value of status as pending
93
94 • ALTER TABLE Orders
95   MODIFY status ENUM('Pending', 'Success', 'Cancel') DEFAULT 'Pending';
96
97 • describe Orders;
98
99
100
101
```

Below the editor is a "Result Grid" showing the table structure:

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	<u>HULL</u>	auto_increment
customer_id	int	YES	MUL	<u>HULL</u>	
product_id	int	YES		<u>HULL</u>	
quantity	int	NO		<u>HULL</u>	
order_date	date	NO		<u>HULL</u>	
status	enum('Pending','Success','Cancel')	YES		Pending	
payment_method	enum('Credit','Debit','UPI')	NO		<u>HULL</u>	
total_amount	decimal(10,2)	NO		<u>HULL</u>	

c) Modify payment_method ENUM to add one more value: 'COD'

- Updated payment method by adding 'COD' option

The screenshot shows a database IDE window titled "06_alter_orders_table". The SQL editor contains the following code:

```
97
98 -- adding new payment option COD in payment_method
99 • ALTER TABLE Orders
100   MODIFY payment_method ENUM('Credit', 'Debit', 'UPI', 'COD');
101
102 • DESCRIBE Orders;
103
104
105
```

Below the editor is a "Result Grid" showing the updated table structure:

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	<u>HULL</u>	auto_increment
customer_id	int	YES	MUL	<u>HULL</u>	
product_id	int	YES		<u>HULL</u>	
quantity	int	NO		<u>HULL</u>	
order_date	date	NO		<u>HULL</u>	
status	enum('Pending','Success','Cancel')	YES		Pending	
payment_method	enum('Credit','Debit','UPI','COD')	YES		<u>HULL</u>	
total_amount	decimal(10,2)	NO		<u>HULL</u>	

d) Make product id as foreign key

- Established foreign key relationship for product_id to maintain data integrity.

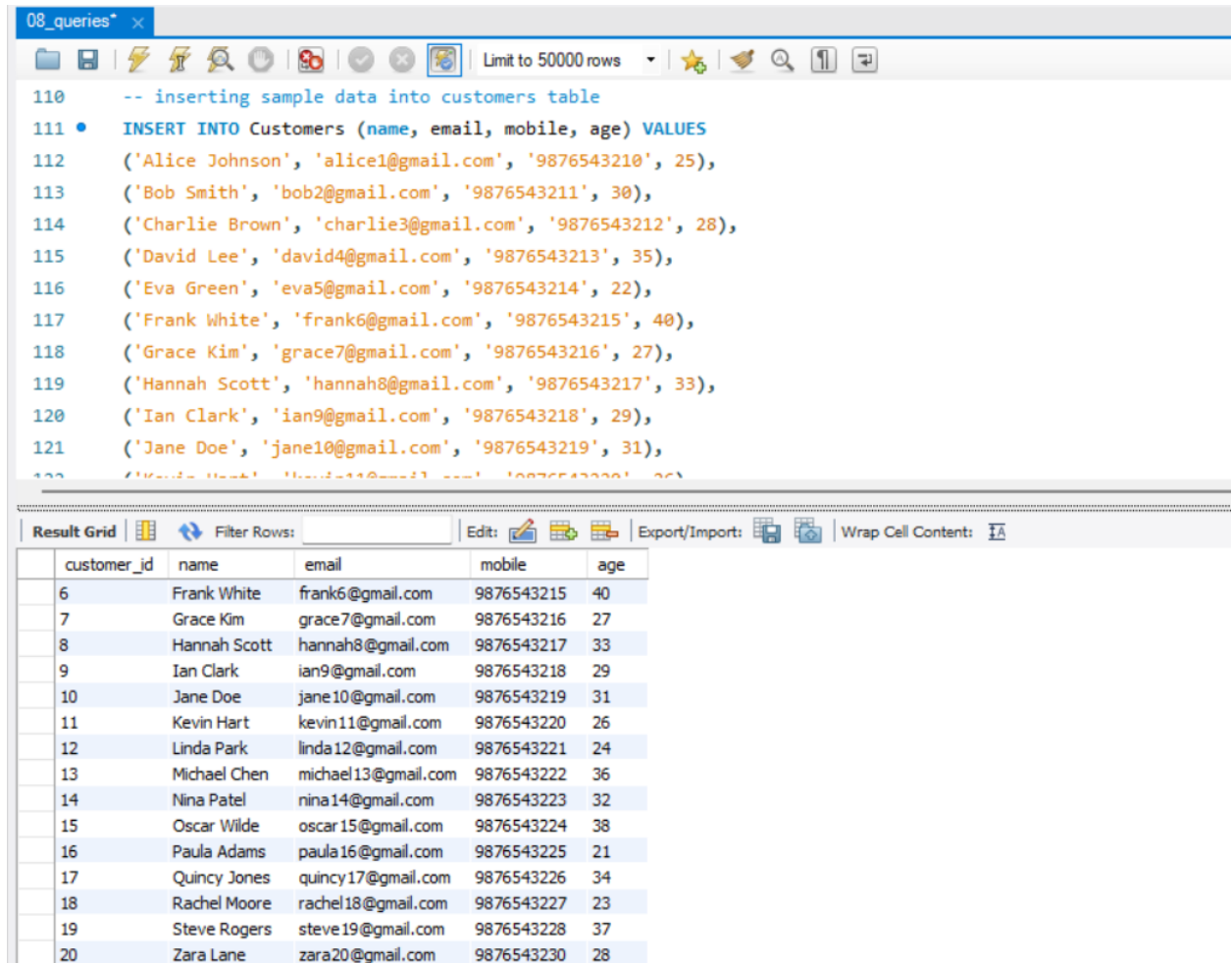
The screenshot shows a database management tool interface. The top pane displays SQL commands for altering the 'Orders' table to add a foreign key constraint for 'product_id' and then describing the table. The bottom pane shows the resulting table structure for 'Orders'.

```
103
104  -- adding foreign key for product_id to maintain relation with products table
105  • ALTER TABLE Orders
106  ADD CONSTRAINT foreign_key_product
107  FOREIGN KEY (product_id) REFERENCES Products(product_id);
108
109  • DESCRIBE Orders;
110
111
```

Field	Type	Null	Key	Default	Extra
order_id	int	NO	PRI	<u>NULL</u>	auto_increment
customer_id	int	YES	MUL	<u>NULL</u>	
product_id	int	YES	MUL	<u>NULL</u>	
quantity	int	NO		<u>NULL</u>	
order_date	date	NO		<u>NULL</u>	
status	enum('Pending','Success','Cancel')	YES		Pending	
payment_method	enum('Credit','Debit','UPI','COD')	YES		<u>NULL</u>	
total_amount	decimal(10,2)	NO		<u>NULL</u>	

6) Insert 20 sample records in all the tables.

- Inserted sample records into all tables.



The screenshot displays a database query editor window titled "08_queries". The editor shows an SQL query to insert 20 sample records into a table named "Customers". The query is as follows:

```
110 -- inserting sample data into customers table
111 • INSERT INTO Customers (name, email, mobile, age) VALUES
112 ('Alice Johnson', 'alice1@gmail.com', '9876543210', 25),
113 ('Bob Smith', 'bob2@gmail.com', '9876543211', 30),
114 ('Charlie Brown', 'charlie3@gmail.com', '9876543212', 28),
115 ('David Lee', 'david4@gmail.com', '9876543213', 35),
116 ('Eva Green', 'eva5@gmail.com', '9876543214', 22),
117 ('Frank White', 'frank6@gmail.com', '9876543215', 40),
118 ('Grace Kim', 'grace7@gmail.com', '9876543216', 27),
119 ('Hannah Scott', 'hannah8@gmail.com', '9876543217', 33),
120 ('Ian Clark', 'ian9@gmail.com', '9876543218', 29),
121 ('Jane Doe', 'jane10@gmail.com', '9876543219', 31),
122 ('Kevin Hart', 'kevin11@gmail.com', '9876543220', 26),
123 ('Linda Park', 'linda12@gmail.com', '9876543221', 24),
124 ('Michael Chen', 'michael13@gmail.com', '9876543222', 36),
125 ('Nina Patel', 'nina14@gmail.com', '9876543223', 32),
126 ('Oscar Wilde', 'oscar15@gmail.com', '9876543224', 38),
127 ('Paula Adams', 'paula16@gmail.com', '9876543225', 21),
128 ('Quincy Jones', 'quincy17@gmail.com', '9876543226', 34),
129 ('Rachel Moore', 'rachel18@gmail.com', '9876543227', 23),
130 ('Steve Rogers', 'steve19@gmail.com', '9876543228', 37),
131 ('Zara Lane', 'zara20@gmail.com', '9876543230', 28);
```

Below the query editor, the "Result Grid" is displayed, showing the 20 inserted records. The grid has columns for customer_id, name, email, mobile, and age.

customer_id	name	email	mobile	age
6	Frank White	frank6@gmail.com	9876543215	40
7	Grace Kim	grace7@gmail.com	9876543216	27
8	Hannah Scott	hannah8@gmail.com	9876543217	33
9	Ian Clark	ian9@gmail.com	9876543218	29
10	Jane Doe	jane10@gmail.com	9876543219	31
11	Kevin Hart	kevin11@gmail.com	9876543220	26
12	Linda Park	linda12@gmail.com	9876543221	24
13	Michael Chen	michael13@gmail.com	9876543222	36
14	Nina Patel	nina14@gmail.com	9876543223	32
15	Oscar Wilde	oscar15@gmail.com	9876543224	38
16	Paula Adams	paula16@gmail.com	9876543225	21
17	Quincy Jones	quincy17@gmail.com	9876543226	34
18	Rachel Moore	rachel18@gmail.com	9876543227	23
19	Steve Rogers	steve19@gmail.com	9876543228	37
20	Zara Lane	zara20@gmail.com	9876543230	28

© 2014 Pearson Education, Inc. or its affiliate(s). All rights reserved. This material is intended solely for the personal, noncommercial use of the individual user. Reproduction or distribution of this material is prohibited by law.

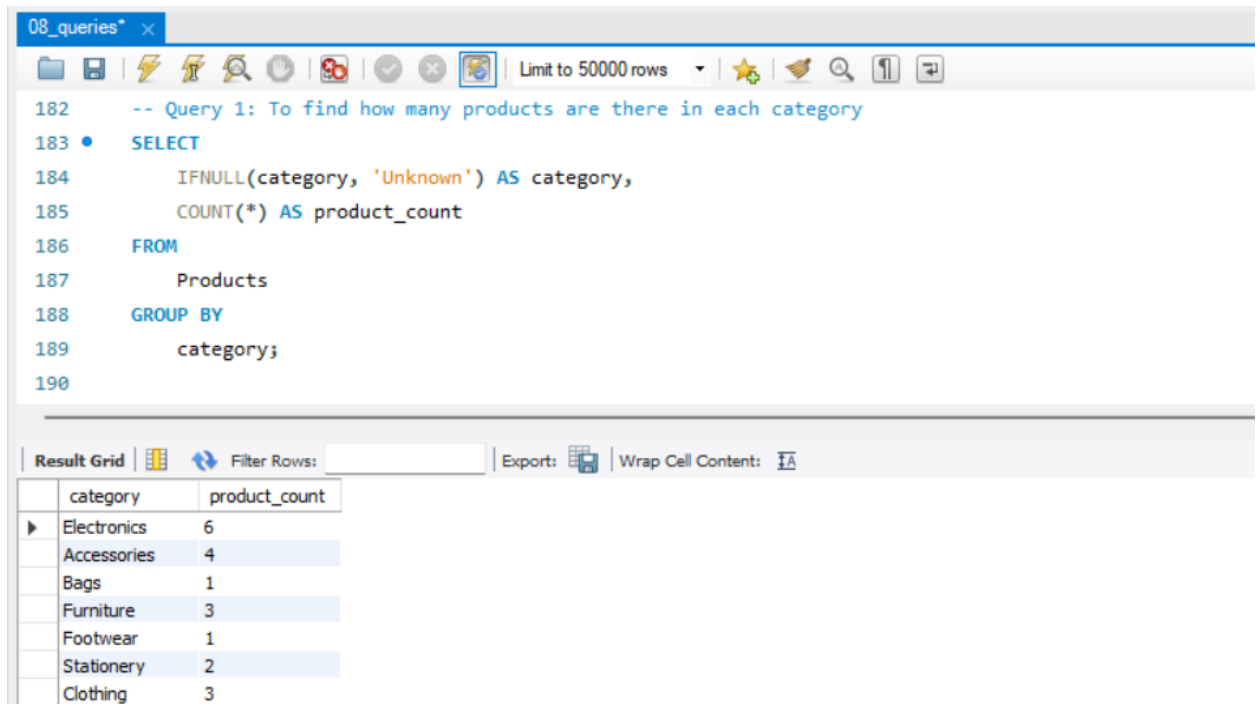
order_id	customer_id	product_id	quantity	order_date	status	payment_method	total_amount
----------	-------------	------------	----------	------------	--------	----------------	--------------

	order_id	customer_id	product_id	quantity	order_date	status	payment_method	total_amount
▶	1	1	1	1	2026-03-01	Pending	UPI	55000.00
	2	2	2	2	2026-03-02	Success	Credit	50000.00
	3	3	3	1	2026-03-03	Cancel	Debit	5000.00
	4	4	4	1	2026-03-04	Success	UPI	3000.00
	5	5	5	3	2026-03-05	Pending	COD	4500.00
	6	6	6	2	2026-03-06	Success	Credit	20000.00
	7	7	7	1	2026-03-07	Pending	UPI	2500.00
	8	8	8	1	2026-03-08	Success	Debit	7000.00
	9	9	9	1	2026-03-09	Cancel	UPI	6000.00
	10	10	10	2	2026-03-10	Success	Credit	8000.00
	11	11	11	1	2026-03-11	Pending	UPI	3500.00
	12	12	12	1	2026-03-12	Success	COD	45000.00
	13	13	13	1	2026-03-13	Success	Credit	12000.00
	14	14	14	5	2026-03-14	Pending	UPI	1000.00
	15	15	15	5	2026-03-15	Success	Debit	750.00
	16	16	16	3	2026-03-16	Pending	UPI	2100.00

7) Perform following queries:

a) Count the number of products as product_count in each category.

- Used GROUP BY to group products based on category and COUNT() to get total in each group.
- Also used IFNULL so that categories with missing values are shown as 'Unknown' for better readability.



The screenshot shows a SQL query editor window titled '08_queries*'. The query is as follows:

```
-- Query 1: To find how many products are there in each category
SELECT
  IFNULL(category, 'Unknown') AS category,
  COUNT(*) AS product_count
FROM
  Products
GROUP BY
  category;
```

Below the query editor is the 'Result Grid' showing the results of the query. The grid has two columns: 'category' and 'product_count'.

category	product_count
Electronics	6
Accessories	4
Bags	1
Furniture	3
Footwear	1
Stationery	2
Clothing	3

b) Retrieve all products that belong to the 'Electronics' category, have a price between \$50 and \$500, and whose name contains the letter 'a'.

- Used WHERE to filter only Electronics products.
- Applied BETWEEN for price range and LIKE to search products containing letter 'a', simulating real search conditions.

08_queries*

Limit to 50000 rows

```

191
192 -- Query 2: To get all Electronics products that cost between 50 and 500 and have 'a' in their name
193 • SELECT
194     *
195 FROM
196     Products
197 WHERE
198     category = 'Electronics'
199     AND price BETWEEN 50 AND 500
200     AND name LIKE '%a%';
201

```

Result Grid

product_id	name	description	price	category
19	Adapter	USB-C adapter	200.00	Electronics
* NULL	NULL	NULL	NULL	NULL

c) Get the top 5 most expensive products in the 'Electronics' category, skipping the first 2.

- Used ORDER BY DESC to sort products from highest to lowest price.

08_queries*

Limit to 50000 rows

```

202 -- Query 3: To Show top 5 expensive Electronics products, skip the first 2
203 -- To Show Electronics products sorted by price from highest to lowest
204 • SELECT
205     *
206 FROM
207     Products
208 WHERE
209     category = 'Electronics'
210 ORDER BY
211     price DESC;
212

```

Result Grid

product_id	name	description	price	category
1	Laptop	Intel i5, 8GB RAM, 512GB SSD	55000.00	Electronics
12	Camera	DSLR 24MP with lens	45000.00	Electronics
2	Smartphone	6.5 inch display, 128GB storage	25000.00	Electronics
13	Printer	Laser printer with WiFi	12000.00	Electronics
6	Monitor	24 inch Full HD	10000.00	Electronics
19	Adapter	USB-C adapter	200.00	Electronics
* NULL	NULL	NULL	NULL	NULL

- Applied LIMIT and OFFSET to skip first few records and fetch only required top results

08_queries* x

Limit to 50000 rows

```

213  -- To Show top 5 expensive Electronics products, skip the first 2
214  • SELECT
215      *
216  FROM
217      Products
218  WHERE
219      category = 'Electronics'
220  ORDER BY
221      price DESC
222  LIMIT 5 OFFSET 2;
223

```

Result Grid

	product_id	name	description	price	category
▶	2	Smartphone	6.5 inch display, 128GB storage	25000.00	Electronics
	13	Printer	Laser printer with WiFi	12000.00	Electronics
	6	Monitor	24 inch Full HD	10000.00	Electronics
	19	Adapter	USB-C adapter	200.00	Electronics
*	NULL	NULL	NULL	NULL	NULL

d) Retrieve customers who have not placed any orders.

- Used LEFT JOIN to include all customers even if they have no orders.
- Then filtered NULL values to find customers who have not placed any orders.
- Useful for identifying inactive users.

08_queries* x

Limit to 50000 rows

```

224  -- Query 4: To show Customers who have not placed any orders
225
226  -- Inserting a new customer who has not placed any orders
227  • INSERT INTO Customers(name, email, mobile, age)
228      VALUES ('Test Student', 'teststudent@email.com', '9999999999', 22);
229
230  -- Using LEFT JOIN to efficiently get customers without any matching orders
231  • SELECT c.*
232  FROM Customers c
233  LEFT JOIN Orders o ON c.customer_id = o.customer_id
234  WHERE o.customer_id IS NULL;

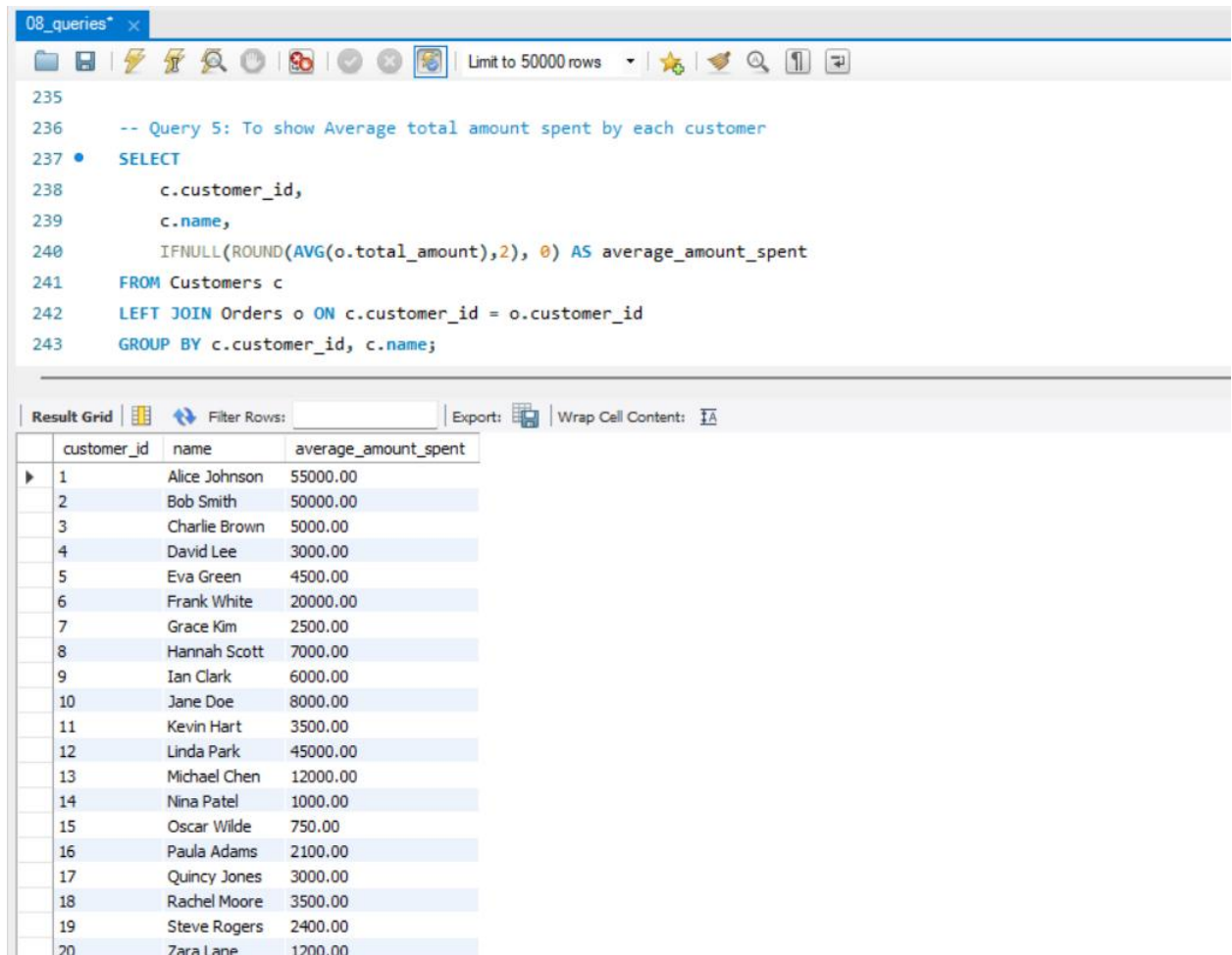
```

Result Grid

	customer_id	name	email	mobile	age
▶	21	Test Student	teststudent@email.com	9999999999	22

e) Find the average total amount spent by each customer.

- Used LEFT JOIN to include all customers in result.
- Applied AVG() to calculate average spending and IFNULL to handle customers with no orders by showing 0
- Helps in analyzing customer purchasing behavior.



The screenshot shows a database query editor window titled "08_queries". The query is as follows:

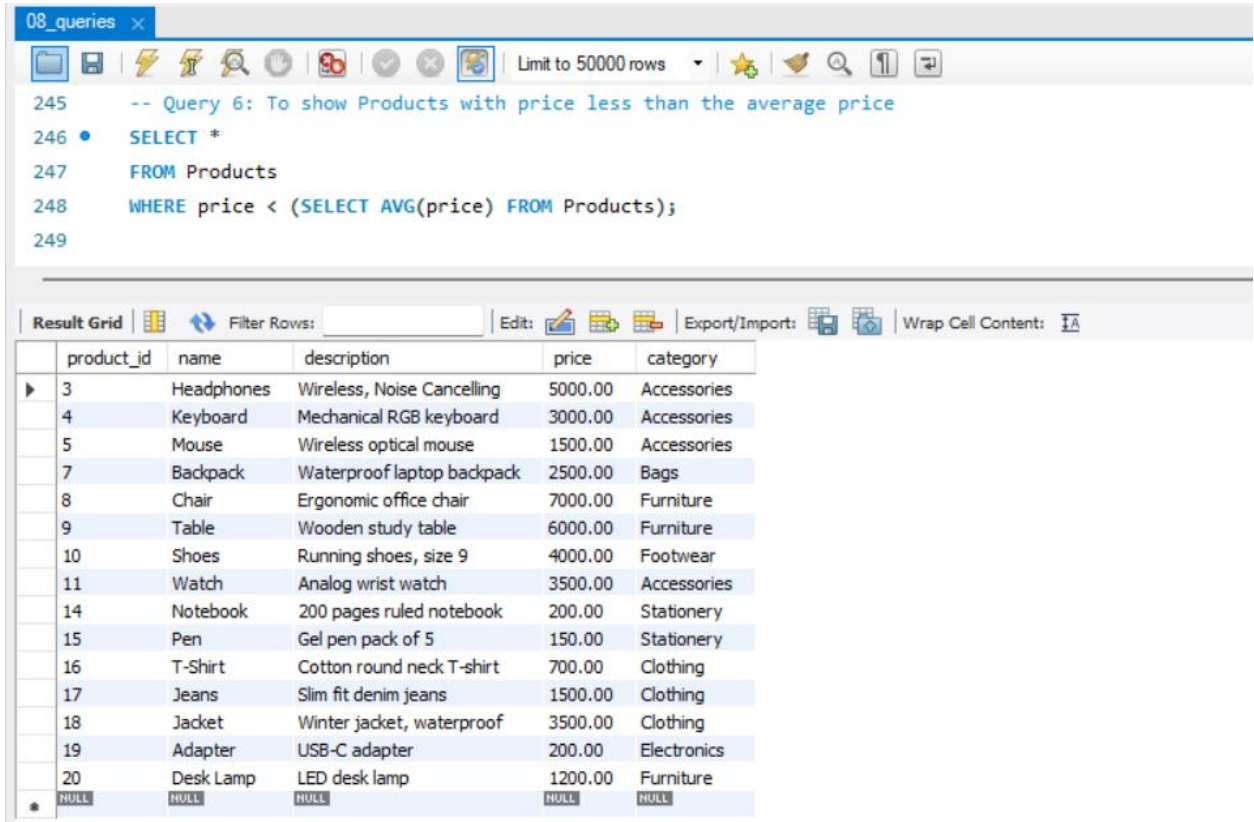
```
-- Query 5: To show Average total amount spent by each customer
SELECT
  c.customer_id,
  c.name,
  IFNULL(ROUND(AVG(o.total_amount),2), 0) AS average_amount_spent
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

The results are displayed in a table with the following columns: customer_id, name, and average_amount_spent. The table contains 20 rows of data.

customer_id	name	average_amount_spent
1	Alice Johnson	55000.00
2	Bob Smith	50000.00
3	Charlie Brown	5000.00
4	David Lee	3000.00
5	Eva Green	4500.00
6	Frank White	20000.00
7	Grace Kim	2500.00
8	Hannah Scott	7000.00
9	Ian Clark	6000.00
10	Jane Doe	8000.00
11	Kevin Hart	3500.00
12	Linda Park	45000.00
13	Michael Chen	12000.00
14	Nina Patel	1000.00
15	Oscar Wilde	750.00
16	Paula Adams	2100.00
17	Quincy Jones	3000.00
18	Rachel Moore	3500.00
19	Steve Rogers	2400.00
20	Zara Lane	1200.00

f) Get the products that have a price less than the average price of all products.

- Used a subquery to calculate overall average price.
- Compared each product's price with this average to find lower-priced products.
- Useful for identifying budget-friendly products.



The screenshot shows a database query editor window titled "08_queries". The query editor contains the following SQL code:

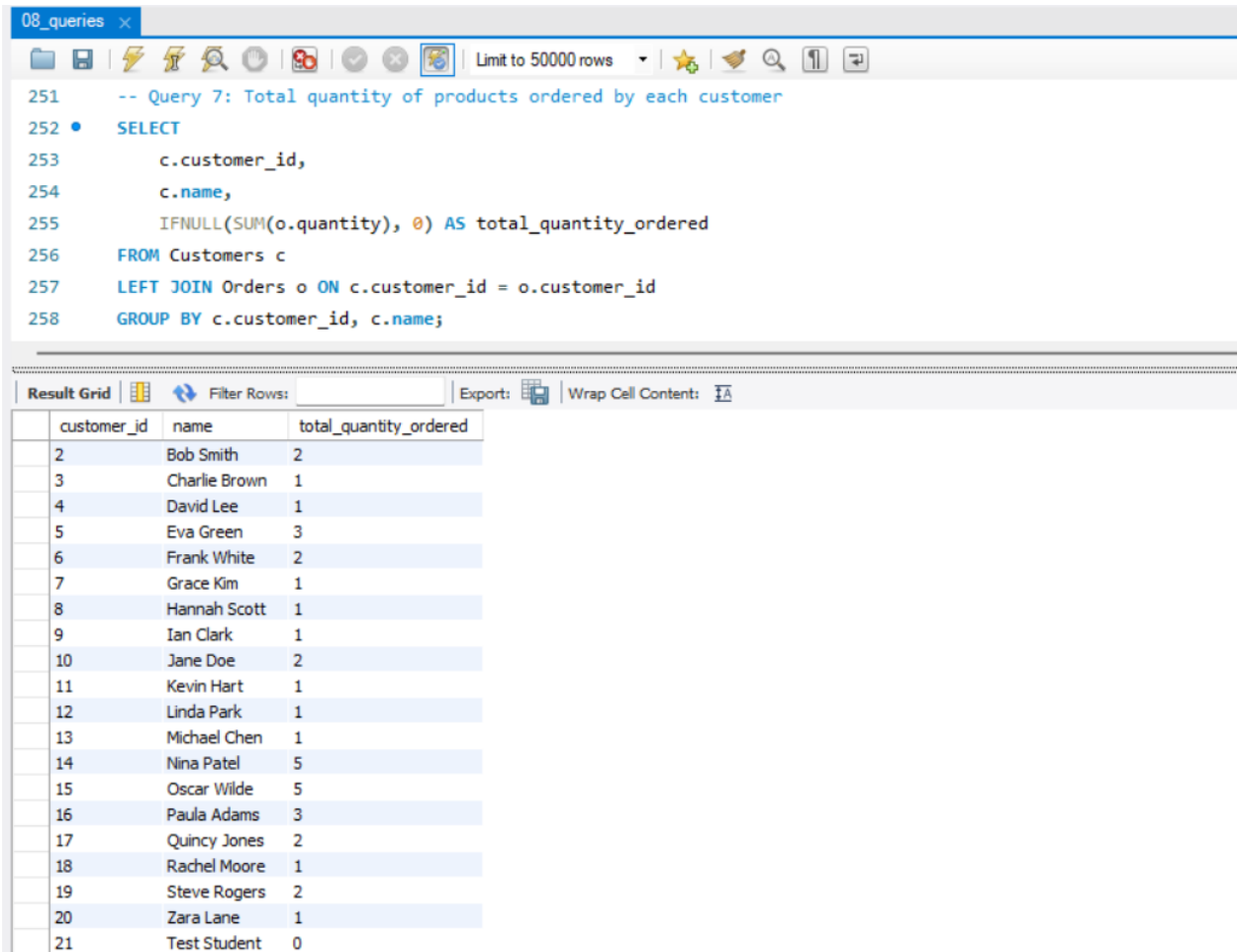
```
-- Query 6: To show Products with price less than the average price
SELECT *
FROM Products
WHERE price < (SELECT AVG(price) FROM Products);
```

Below the query editor, the "Result Grid" displays the results of the query. The results are shown in a table with the following columns: product_id, name, description, price, and category. The table contains 20 rows of data, with the last row showing NULL values for all columns.

product_id	name	description	price	category
3	Headphones	Wireless, Noise Cancelling	5000.00	Accessories
4	Keyboard	Mechanical RGB keyboard	3000.00	Accessories
5	Mouse	Wireless optical mouse	1500.00	Accessories
7	Backpack	Waterproof laptop backpack	2500.00	Bags
8	Chair	Ergonomic office chair	7000.00	Furniture
9	Table	Wooden study table	6000.00	Furniture
10	Shoes	Running shoes, size 9	4000.00	Footwear
11	Watch	Analog wrist watch	3500.00	Accessories
14	Notebook	200 pages ruled notebook	200.00	Stationery
15	Pen	Gel pen pack of 5	150.00	Stationery
16	T-Shirt	Cotton round neck T-shirt	700.00	Clothing
17	Jeans	Slim fit denim jeans	1500.00	Clothing
18	Jacket	Winter jacket, waterproof	3500.00	Clothing
19	Adapter	USB-C adapter	200.00	Electronics
20	Desk Lamp	LED desk lamp	1200.00	Furniture
NULL	NULL	NULL	NULL	NULL

g) Calculate the total quantity of products ordered by each customer:

- Used SUM() to calculate total quantity ordered.
- Used LEFT JOIN to include customers without orders and IFNULL to show 0 instead of NULL.



The screenshot shows a database query editor window titled "08_queries". The query is as follows:

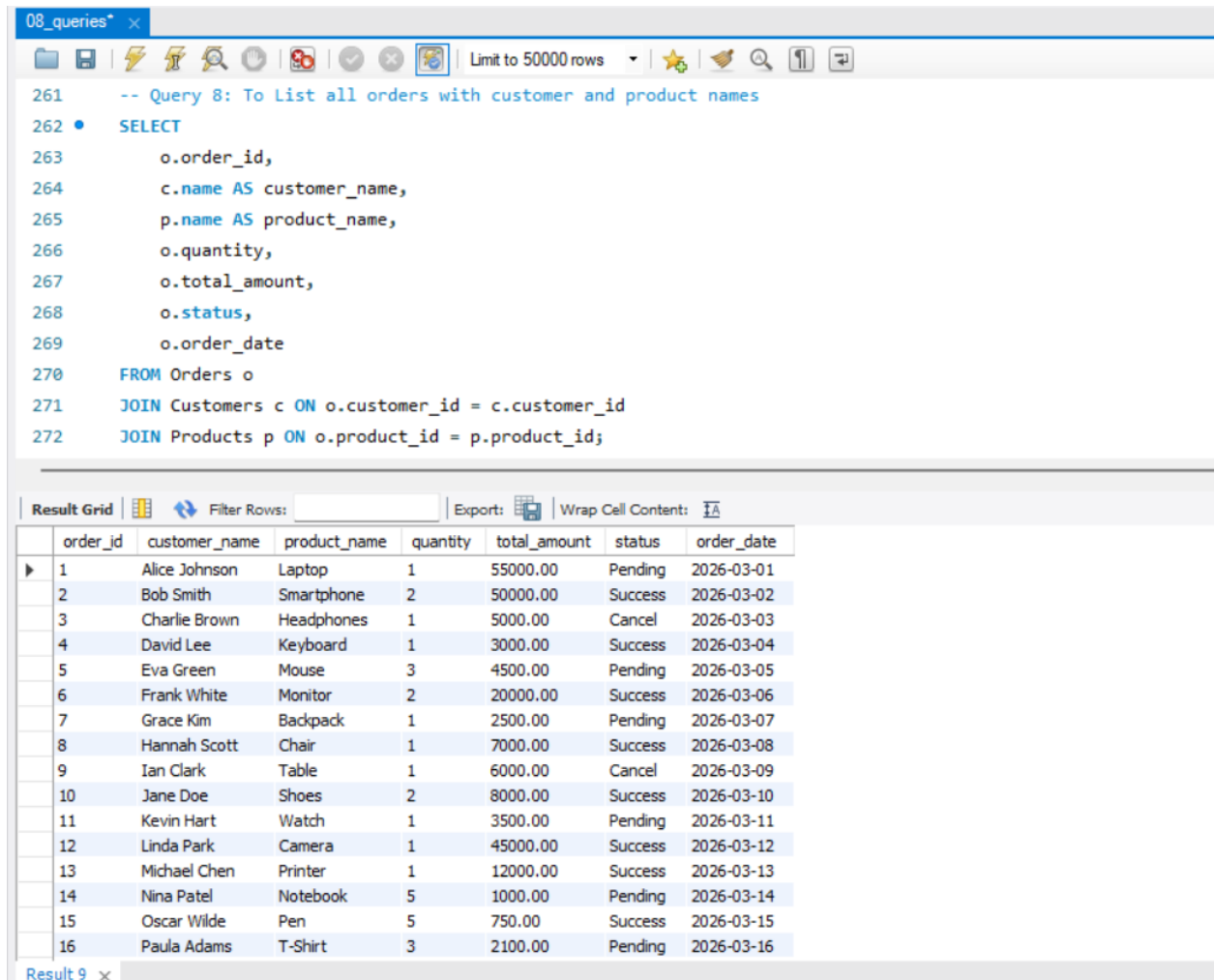
```
-- Query 7: Total quantity of products ordered by each customer
SELECT
  c.customer_id,
  c.name,
  IFNULL(SUM(o.quantity), 0) AS total_quantity_ordered
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

Below the query editor, the "Result Grid" is displayed, showing the results of the query. The grid has three columns: "customer_id", "name", and "total_quantity_ordered". The results are as follows:

customer_id	name	total_quantity_ordered
2	Bob Smith	2
3	Charlie Brown	1
4	David Lee	1
5	Eva Green	3
6	Frank White	2
7	Grace Kim	1
8	Hannah Scott	1
9	Ian Clark	1
10	Jane Doe	2
11	Kevin Hart	1
12	Linda Park	1
13	Michael Chen	1
14	Nina Patel	5
15	Oscar Wilde	5
16	Paula Adams	3
17	Quincy Jones	2
18	Rachel Moore	1
19	Steve Rogers	2
20	Zara Lane	1
21	Test Student	0

h) List all orders along with customer name and product name.

- Used INNER JOIN to combine Orders, Customers, and Products tables.
This ensures only valid and matching records are displayed.
- Retrieved order details along with corresponding customer and product names.



The screenshot shows a database query editor window titled "08_queries". The query is as follows:

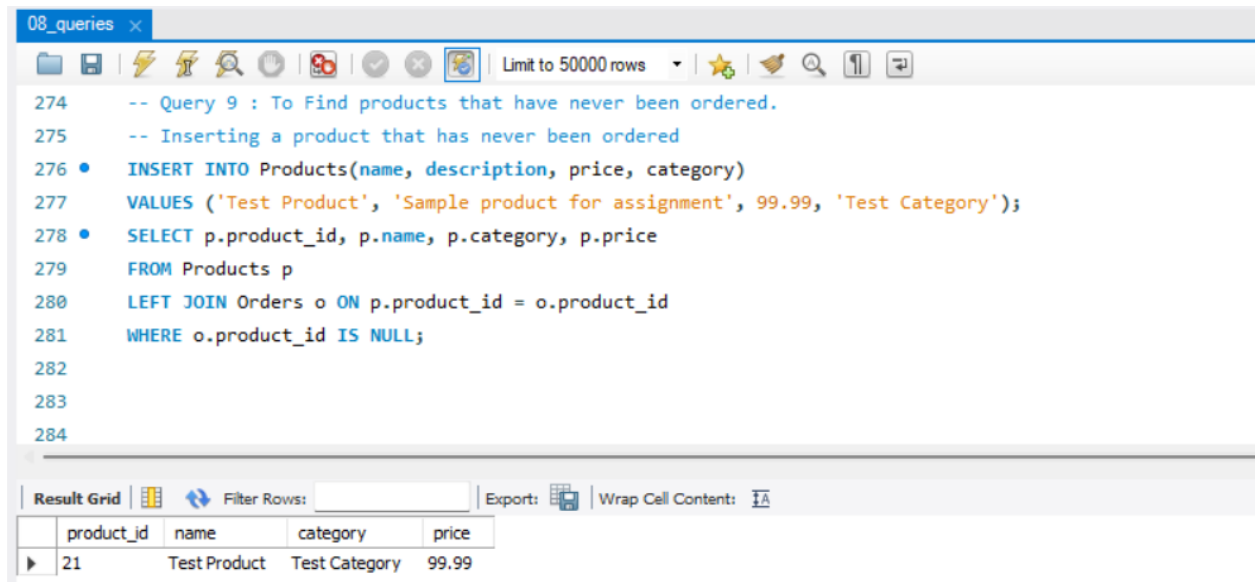
```
-- Query 8: To List all orders with customer and product names
SELECT
    o.order_id,
    c.name AS customer_name,
    p.name AS product_name,
    o.quantity,
    o.total_amount,
    o.status,
    o.order_date
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Products p ON o.product_id = p.product_id;
```

Below the query, the "Result Grid" shows 16 rows of data. The columns are: order_id, customer_name, product_name, quantity, total_amount, status, and order_date.

order_id	customer_name	product_name	quantity	total_amount	status	order_date
1	Alice Johnson	Laptop	1	55000.00	Pending	2026-03-01
2	Bob Smith	Smartphone	2	50000.00	Success	2026-03-02
3	Charlie Brown	Headphones	1	5000.00	Cancel	2026-03-03
4	David Lee	Keyboard	1	3000.00	Success	2026-03-04
5	Eva Green	Mouse	3	4500.00	Pending	2026-03-05
6	Frank White	Monitor	2	20000.00	Success	2026-03-06
7	Grace Kim	Backpack	1	2500.00	Pending	2026-03-07
8	Hannah Scott	Chair	1	7000.00	Success	2026-03-08
9	Ian Clark	Table	1	6000.00	Cancel	2026-03-09
10	Jane Doe	Shoes	2	8000.00	Success	2026-03-10
11	Kevin Hart	Watch	1	3500.00	Pending	2026-03-11
12	Linda Park	Camera	1	45000.00	Success	2026-03-12
13	Michael Chen	Printer	1	12000.00	Success	2026-03-13
14	Nina Patel	Notebook	5	1000.00	Pending	2026-03-14
15	Oscar Wilde	Pen	5	750.00	Success	2026-03-15
16	Paula Adams	T-Shirt	3	2100.00	Pending	2026-03-16

i) Find products that have never been ordered.

- Used LEFT JOIN to include all products.
- Filtered NULL values to find products that have no matching records in Orders.
- Helps in analyzing unsold inventory



```
08_queries x
Limit to 50000 rows
274 -- Query 9 : To Find products that have never been ordered.
275 -- Inserting a product that has never been ordered
276 • INSERT INTO Products(name, description, price, category)
277   VALUES ('Test Product', 'Sample product for assignment', 99.99, 'Test Category');
278 • SELECT p.product_id, p.name, p.category, p.price
279   FROM Products p
280  LEFT JOIN Orders o ON p.product_id = o.product_id
281  WHERE o.product_id IS NULL;
282
283
284
```

Result Grid

	product_id	name	category	price
▶	21	Test Product	Test Category	99.99

Note: This assignment was also maintained using Git version control with a separate SQL branch and structured commits.

GitHub Repository : <https://github.com/Shinu87/nucleusteq-fresher-training>